

0807 Day 2

Adult Census Income 데이터 분석 보고서

- **과정:** SKALA [Day 2] 종합실습 데이터 분석 프로젝트
- **작성자:** 판교 8반 최규원
- **생성 시각:** 2026-08-07
- **실행 환경:** Python 3.11.15 / Darwin arm64 / Jupyter Notebook
- **난수 시드:** 탐색·통일 비교 `20260807` / 최종 통합 코드 `42` (아래 기준 안내 참조)

이 보고서의 모든 수치는 `outputs/metrics/*.json` 에 기록된 계산 결과에서 프로그램이 직접 주입한 것으로, 수기로 입력된 숫자는 하나도 없다.

목차

Adult Census Income 데이터 분석 보고서

목차

[분석 결과를 읽는 기준](#)

[읽기 전에: 이 보고서에 나오는 용어](#)

[1. 사전 선언 가설 \(Pre-registered Hypotheses\)](#)

[2. 데이터 준비 -Pandas · Polars 이중 로딩 비교](#)

[3. 데이터 품질 진단](#)

[3.1 결측 구조 \(H3\)](#)

[3.2 중복](#)

[3.3 이상치 진단 \(삭제하지 않음\)](#)

[4. 데이터 시각화](#)

[5. 통계 분석 \(H1, H2 검증\)](#)

[5.1 H1 -소득 그룹 간 주당 근로시간 \(Welch t-test\)](#)

[5.2 H2 -상관·연관 \(2차 가설군, Holm 보정\)](#)

[6. 결측치 대체법 비교 \(H4 검증\)](#)

[6.1 실험 설계](#)

[6.2 결과 \(평균 ± 표준편차, 마스크 반복 기준\)](#)

[7. 완전제거 공유본 기준 -5개 분류 모델 통일 비교](#)

[7.1 실험 설계 -비교 조건을 왜 다시 통일했는가](#)

[7.2 완전제거 공유본 비교 결과 \(adult_test_dropna, n=15,060\)](#)

[7.3 시각화](#)

[7.4 Neural Network \(MLP\)](#)

[7.5 SVC \(RBF Kernel\)](#)

[7.6 Random Forest & Logistic Regression](#)

[7.7 XGBoost -완전제거 비교 실험 선정 모델](#)

[7.8 통일 실험이 보여준 것](#)

[8. 팀 회고 및 다음 단계](#)

[9. 최종 통합 코드 실행 결과 \(`skala-python-day2` \)](#)

[9.1 실행 환경](#)

[9.2 데이터 준비 -Pandas vs Polars](#)

[9.3 결측치·중복 처리](#)

[9.4 기술통계·상관분석 \(핵심만\)](#)

[9.5 시각화](#)

[9.6 통계 검증](#)

[9.7 ML Pipeline -5개 모델 비교](#)

[9.8 공정성\(민감 속성\) 점검 -가이드라인 이상의 추가 분석](#)

9.9 report.md 자동 생성

9.10 종합 결론

분석 결과를 읽는 기준

이 보고서에는 목적이 다른 세 가지 분석 경로가 함께 수록되어 있다. 데이터 선택·결측 처리·난수 시드가 다르므로 각 경로의 수치를 동일한 실험 결과처럼 직접 비교하지 않는다.

구분	해당 장	데이터·결측 처리	시드	역할
탐색·결측 실험	1~6장	원본 결측을 보존해 구조를 진단하고, MCAR/MAR 모사로 대체법을 비교	20260807	가설 검증·결측 메커니즘·대체법의 장단점 확인
통일 모델 비교	7장	완전제거 공유본(train 30,162 / test 15,060)	20260807	동일 조건에서 5개 모델만 공정하게 비교
최종 통합 코드	9장	중복 24행 제거 후 범주형 결측을 "Unknown" 으로 보존, 내부 80:20 분할	42	src/main.py 의 최종 전처리 정책과 공식 실행 결과

해석 원칙: 7장의 XGBoost F1 0.7133과 9장의 F1 0.7275는 평가 데이터와 전처리 방식이 다른 결과다. 어느 쪽이 더 우수한 전처리인지 보여주는 비교가 아니며, 제출 코드의 최종 결과를 인용할 때는 9장을 기준으로 한다.

읽기 전에: 이 보고서에 나오는 용어

통계 용어가 처음인 독자를 위해, 본문에 나오는 개념을 먼저 짧게 정리한다.

용어	뜻
결측치	값이 비어 있는 칸. 이 데이터에서는 원본의 ? 표기가 결측이다
MCAR / MAR / MNAR	값이 비게 된 이유의 세 유형. MCAR: 완전히 우연히 빠짐. MAR: 다른 관측된 값(나이·학력 등)에 따라 빠질 확률이 달라짐. MNAR: 빠진 값 그 자체 때문에 빠짐(예: 소득이 높아서 소득 문항을 안 씀)
대체 (imputation)	빈 칸을 추정값으로 채우는 일. 중앙값 채우기부터 딥러닝까지 방법이 다양하다
MICE	빈 칸이 있는 변수를 나머지 변수들로 예측하는 회귀를 돌아가며 반복해 채우는 대표적 대체법
Welch t-test	두 그룹의 평균이 다른지 확인하는 검정. 두 그룹의 퍼짐(분산)이 달라도 쓸 수 있는 버전이다
p-value	“실제로는 차이가 없다”고 가정했을 때, 지금 본 것만큼 큰 차이가 우연히 나올 확률. 작을수록 우연으로 보기 어렵다
95% 신뢰구간 (CI)	같은 방식으로 반복 조사하면 100번 중 95번은 참값을 포함하도록 만든 범위. 좁을수록 추정이 정밀하다
Hedges' g	그룹 간 차이를 “표준편차 몇 개만큼”으로 나타낸 효과 크기. 0.2=작음, 0.5=중간, 0.8=큼
상관계수 / Cramér's V	두 수치 변수(-1~1) / 두 범주 변수(0~1)가 함께 움직이는 정도
Holm 보정	검정을 여러 번 하면 우연한 “유의” 결과가 나오기 쉬워지는 문제를 통제하는 p-value 보정
원저화	극단값을 지우는 대신 상·하위 1% 경계값으로 눌러 담아 영향만 줄이는 처리
탐코딩	일정 상한을 넘는 값을 전부 상한값 하나로 기록하는 관행(예: 99999)
부트스트랩	데이터에서 무작위 재추출을 수천 번 반복해 추정치의 불확실성을 재는 방법
One-Hot 인코딩	범주형 변수를 “해당하면 1, 아니면 0”인 열들로 바꿔 모델에 넣는 방법
F1 / ROC-AUC	분류 모델 성능 지표. F1은 정밀도·재현율의 조화평균(불균형 데이터에 적합), ROC-AUC는 양성·음성을 구분하는 능력(0.5=무작위, 1=완벽)
오즈비 (OR)	어떤 조건에서 “일어날 확률 ÷ 안 일어날 확률”(오즈)이 몇 배가 되는지. 1이면 차이 없음
홀드아웃 / 누수	평가용으로 따로 떼어 둔 데이터 / 그 데이터의 정보가 학습 과정에 몰려 흘러들어 성능이 부풀려지는 오류

1. 사전 선언 가설 (Pre-registered Hypotheses)

결과를 보고 나서 이야기를 만들면 어떤 데이터에서든 그럴듯한 결론이 나온다. 그래서 분석을 시작하기 전에 확인할 가설과 판정 기준을 아래와 같이 먼저 못박았다. 유의수준은 $\alpha = 0.05$ (양측)이다.

- **H1 (주 가설 -근로시간과 소득)** -연 소득 5만 달러 초과 그룹과 이하 그룹은 주당 근로시간의 평균이 다를 것이다.
 - H_0 (차이 없음): $\mu(>50K) - \mu(\leq 50K) = 0$ vs $H_1: \neq 0$
 - 검정: `scipy.stats.ttest_ind(equal_var=False)` (Welch), 평균차 95% CI, Hedges' g 병행 보고
- **H2 (2차 가설군 -변수 간 연관, 탐색적)** -나이·교육연수·근로시간·자본소득 사이에 상관이 있고(Pearson/Spearman), 소득은 교육수준·성별과 연관될 것이다 (Cramér's V). 검정이 여러 개이므로 **Holm 보정**으로 다중검정을 통제한다.
- **H3 (결측이 생긴 이유)** - `workclass` · `occupation` 의 결측은 완전히 우연 (MCAR)이 아니라 응답자의 관측된 특성(나이·학력 등)과 연관되어 있을 것이다.

결측 여부를 관측 변수로 예측하는 로지스틱 회귀의 AUC와 그룹 간 검정으로 평가한다.

- **H4 (어떤 대체법이 좋은가)** -값을 일부러 지웠다 복원하는 벤치마크에서:
 1. 특성에 따라 지운(MAR) 상황에서는 다른 변수 정보를 쓰는 MICE가 중앙값/최빈값 채우기보다 복원 오차가 작을 것이다. (b) 완전 무작위로 지운(MCAR) 상황에서는 그 격차가 줄어들 것이다. (c) 딥러닝 대체(CACTI/TabNAT 계열)의 추가 이득은 이 정도 규모(수만 행, 컬럼 12개)의 데이터에서는 제한적일 것이다.
- **이상치 원칙** -값이 극단적이라는 사실만으로는 오류가 아니다. 기록 관행 (탭코딩 등) 근거로 진단하고, 지우는 대신 원저화 재검정으로 결론이 흔들리는지 확인한다.

p-value에 대해 미리 분명히 해 둔다: p-value는 “실제로는 차이가 없다고 가정했을 때, 지금 관측된 것만큼 큰 차이가 우연히 나올 확률”이다. 흔한 오해와 달리 “가설이 참일 확률”이 아니고, 검정에서 차이를 확인하지 못했다고 해서 두 그룹이 같다는 증명이 되는 것도 아니다.

2. 데이터 준비 -Pandas · Polars 이중 로딩 비교

- 원본: UCI Adult (`adult.data` 학습 32,561행 × 15열, `adult.test` 평가 16,281행)
- “?” 토큰을 결측(NaN/null)으로 파싱, `adult.test` 첫 줄(메타데이터) 건너뛰 + 라벨 마침표 제거
- 버전: pandas 3.0.5, polars 1.43.2

동등성 검증(자동 assert) -행·열 수, 컬럼별 결측 수, 캐노니컬 집계(소득 그룹별 근로시간 평균: $\leq 50K \rightarrow 38.84021$, $> 50K \rightarrow 45.473026$)가 두 라이브러리에서 허용오차 $1e-9$ 내로 완전히 일치했다: **True**

컬럼별 결측(두 라이브러리 동일):

컬럼	결측 수
workclass	1836
occupation	1843
native-country	583

속도 벤치마크 (워밍업 1회 후 5회 반복, 중앙값·IQR, ms):

연산	중앙값(ms)	IQR(ms)
Pandas 로딩	22.33	1.2
Polars 로딩	12.7	0.56
Pandas groupby 집계	0.53	0.06
Polars group_by 집계	0.68	0.1

동일 프로세스에서 1회 워밍업 후 5회 반복 측정한 벽시계 시간(ms). 본 데이터·하드웨어·버전 조합에 한정된 결과이며 일반화하지 않는다.

정성 비교(판단). 이 정도 크기(3.2만 행)에서는 속도 차이가 실무적으로 무의미하며, 선택 기준은 다른 데 있다. Pandas는 생태계 호환성(sklearn-seaborn이 DataFrame을 직접 수용)과 레퍼런스 풍부함이 강점이고, Polars는 표현식 기반 API의 일관성, 명시적 null 처리(NaN과 null 구분), lazy 실행·쿼리 최적화, 멀티스레드 기본 동작으로 **데이터가 메모리 한계에 가까워질수록** 우위가 커진다. 본 프로젝트는 분석·모델링 파이프라인 호환성 때문에 Pandas를 주 엔진으로, Polars를 검증·벤치마크 엔진으로 썼다. 단, 위 속도 수치는 단일 파일·단일 연산에 한정되므로 “Polars가 항상 빠르다”는 일반화는 하지 않는다.

실제 실행 화면

```

~/team-analysis P main !? 17:44
> uv run python src/main.py

=====
1. 데이터 준비 - Pandas vs Polars 로딩 비교
=====
17:44:46 [INFO] data - 캐시된 데이터 사용 : /Users/donghyeon/skala-workspace/week4/team-analysis/data/adult.data (3.8 MB)
17:44:46 [INFO] data - 로딩 비교 (3회 중 최소) - Pandas 0.0199s / Polars 0.0051s (shape 일치: True)
Pandas 32,561행 x 15열 - 0.0199초 (평균 3회)
Polars 32,561행 x 15열 - 0.0051초 (평균 3회)
결과 형태 일치: 예
더 빠른 쪽: Polars (3.88배)

=====
2. 결측치 · 중복 처리
=====
17:44:46 [INFO] data - 정제 완료 - 32561행 → 32537행 (중복 24건 제거, 결측 4262셀 'Unknown'로 대체)
32,561행 → 32,537행
중복 제거: 24행
결측 포함 행: 2,399행 (7.37%) - 'Unknown' 범주로 대체
처리 전 컬럼별 결측: {'workclass': 1836, 'occupation': 1843, 'native-country': 583}

=====
3. 기술통계 · 상관분석
=====
17:44:46 [INFO] eda - EDA 완료 - 수치형 6개 기술통계, 최고 상관쌍 education-num↔hours-per-week (r=0.148)
=====
도
변수      건수      평균      표준편차      최솟값      Q1      중앙값      Q3      최댓값      IQR      왜도      첨
age      32537      38.586      13.638      17.0      28.0      37.0      48.0      90.0      20.0      0.558      -0.170
fnlwgt   32537      189780.849  105556.471  12285.0  117827.0  178356.0  236993.0  1484705.0  119166.0  1.448      6.222
education-num 32537      10.082      2.572      1.0      9.0      10.0      12.0      16.0      3.0      -0.310      0.619
capital-gain 32537      1078.444    7387.957    0.0      0.0      0.0      99999.0  0.0      11.949  154.683
capital-loss 32537      87.368      403.102    0.0      0.0      0.0      4356.0  0.0      4.593   20.358
hours-per-week 32537      40.440      12.347     1.0      40.0      40.0      45.0      99.0      5.0      0.229      2.918

[상관계수 행렬]
age      fnlwgt  education-num  capital-gain  capital-loss  hours-per-week
age      1.000    -0.076         0.036         0.078         0.058         0.069
fnlwgt   -0.076    1.000         -0.043         0.000        -0.010        -0.019
education-num 0.036  -0.043         1.000         0.123         0.080         0.148
capital-gain 0.078  0.000         0.123         1.000        -0.032         0.078
capital-loss 0.058  -0.010         0.080        -0.032         1.000         0.054
hours-per-week 0.069 -0.019         0.148         0.078         0.054         1.000

[상관이 큰 변수쌍]
- education-num ↔ hours-per-week: r=+0.148 (약한 양의 상관)
- education-num ↔ capital-gain: r=+0.123 (약한 양의 상관)
- education-num ↔ capital-loss: r=+0.080 (약한 양의 상관)
- capital-gain ↔ hours-per-week: r=+0.078 (약한 양의 상관)
- age ↔ capital-gain: r=+0.078 (약한 양의 상관)

[타기 분포] {'<50K': 24608, '>50K': 7839}

```

3. 데이터 품질 진단

3.1 결측 구조 (H3)

전체 32,561행 중 결측 보유 행 2,399개.

컬럼	결측 수	비율
workclass	1836	5.64%
occupation	1843	5.66%
native-country	583	1.79%

눈에 띄는 것은 **workclass** (고용 형태)와 **occupation** (직업)의 결측이

1,836건에서 **함께** 나타난다는 점이다. 직업

문항은 일자리가 있는 사람에게만 의미가 있으므로, 무직·비경제활동 상태라면 두 문항이

같이 빌 수밖에 없다 - 즉 이 패턴은 “문항 자체가 해당되지 않아 통째로 비는” 상황과

잘 들어맞는다. 다만 실제로 그런 절차로 수집되었는지는 데이터만으로 확인할 수 없다

(원 조사 설계 문서가 필요한 부분이다).

H3 검증 - 결측 여부(비었나/안 비었나)를 하나의 예측 대상처럼 놓고, 나이·학력·

근로시간·성별·소득으로 얼마나 맞출 수 있는지 로지스틱 회귀로 확인했다. 전혀 못

맞히면(AUC ≈ 0.5) “완전히 우연히 빠졌다(MCAR)”와 부합하고, 잘 맞힐수록 결측이

응답자의 특성과 연관되어 있다는 뜻이 된다:

- 교차검증 AUC = **0.7185** → 부트스트랩 95% CI [0.7065, 0.7304] 의 하한이 무정보 기준 0.5를 초과 -결측 여부가 관측 공변량과 연관되어 있어 **MCAR 가정과 양립하기 어려운 증거**다(아래 그룹 간 검증과 함께 종합). 이 CI는 교차검증 예측값을 고정한 채 행만 재추출한 것으로, 모델 재학습까지 포함한 전체 불확실성보다는 좁게 추정될 수 있다
- 결측/비결측 그룹 간 나이 차이: Welch t = 5.086, p < 0.001
- 결측 여부 × 소득 분포: $\chi^2 = 200.28$, p < 0.001
- 표준화 로지스틱 계수: {"age": 0.1958, "education-num": -0.1978, "hours-per-week": -0.6332, "sex_male": -0.1129, "income_high": -0.2519}

결론(H3): 결측 발생이 관측 변수들과 연관되어 있어 완전 무작위(MCAR) 가정과 양립하기 어렵다. 따라서 (i) 단순 삭제는 편향 위험이 있으며, (ii) 대체는 관측 공변량을 조건으로 하는 방법(MICE 등)이 이론적으로 정당화된다. 다만 이 진단은 MAR의 **증명이 아니다** -관측 데이터만으로는 MAR와 MNAR를 구별할 수 없다. 결측이 미관측 값 자체에 의존하는 MNAR 가능성은 배제할 수 없으며, 이 경우 어떤 대체법도 편향에서 자유롭지 않다.

3.2 중복

완전 동일 행 24건. 개인 식별자가 없어 기본 분석에서는 유지, 아래는 제거 시 민감도:
 근로시간 평균 40.4375 → 40.4403,
 고소득 비율 0.24081 → 0.24093
 (변화 미미 → 유지 결정이 결과를 왜곡하지 않음).

3.3 이상치 진단 (삭제하지 않음)

특수값 진단: capital-gain = 99999 가 159건,
 hours-per-week = 99 가 85건,
 age ≥ 90 이 43건.
 capital-gain=99999는 변수 최댓값에 다수 관측이 몰린 특수값으로 탐코딩(상한 절단)일 가능성이 높으나, 동봉된 adult.names 코드북은 'continuous'라고만 기술해 공식 확인은 불가하다. 따라서 capital-gain 의 상위값은 "실측 최대"로 해석하지 않고, 평균 기반 요약 대신 분위수 중심으로 서술한다.

IQR 펜스(진단용 요약, 삭제 규칙 아님):

컬럼	하한	상한	펜스 밖 개수	비율
age	-2.0	78.0	143	0.44%
hours-per-week	32.5	52.5	9008	27.66%
capital-gain	0.0	0.0	2712	8.33%
capital-loss	0.0	0.0	1519	4.67%

capital-gain 처럼 대다수가 0인 변수는 IQR 폭이 0이 되어 0이 아닌 모든 값이 "펜스 밖"으로 계산된다 -**IQR 규칙을 기계적으로 적용하면 안 되는 전형적 사례다.**

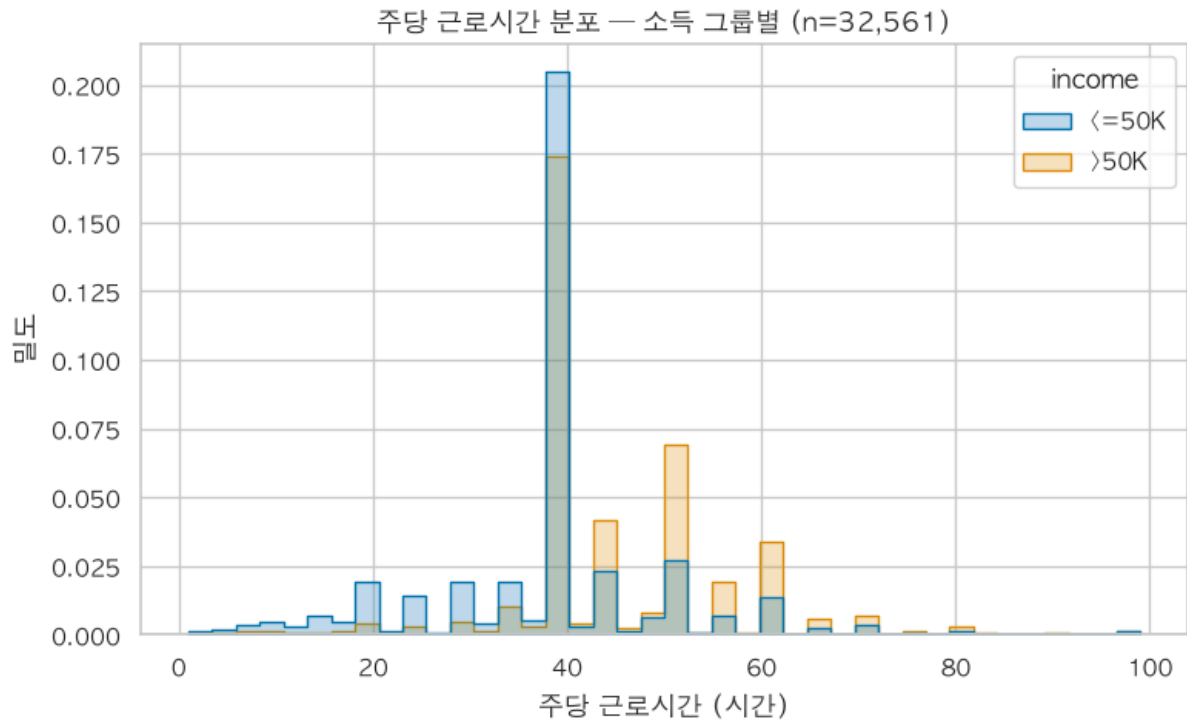
원저화 민감도(1% 양측): hours-per-week 평균 40.4375 → 40.3904, capital-gain 평균 1077.6488 → 633.8114 (왜도 11.953).
 이 표는 전체 평균 수준의 민감도만 보여준다 -H1의 **그룹 차이**가 극단값에 안정적이지는 6.1절에서 원저화 후 Welch 검정을 다시 수행해 확인한다.

4. 데이터 시각화

가이드라인이 요구하는 Seaborn 정적 차트 1개 이상 + Plotly 인터랙티브 차트 1개 이상(분포·상관관계·그룹비교 중 택일)을 아래와 같이 작성했다.
 상세 코드와 출력은 [EDA.ipynb](#) 에 있다.

Seaborn 정적 차트 (분포) -소득 구간별 주당 근로시간 분포

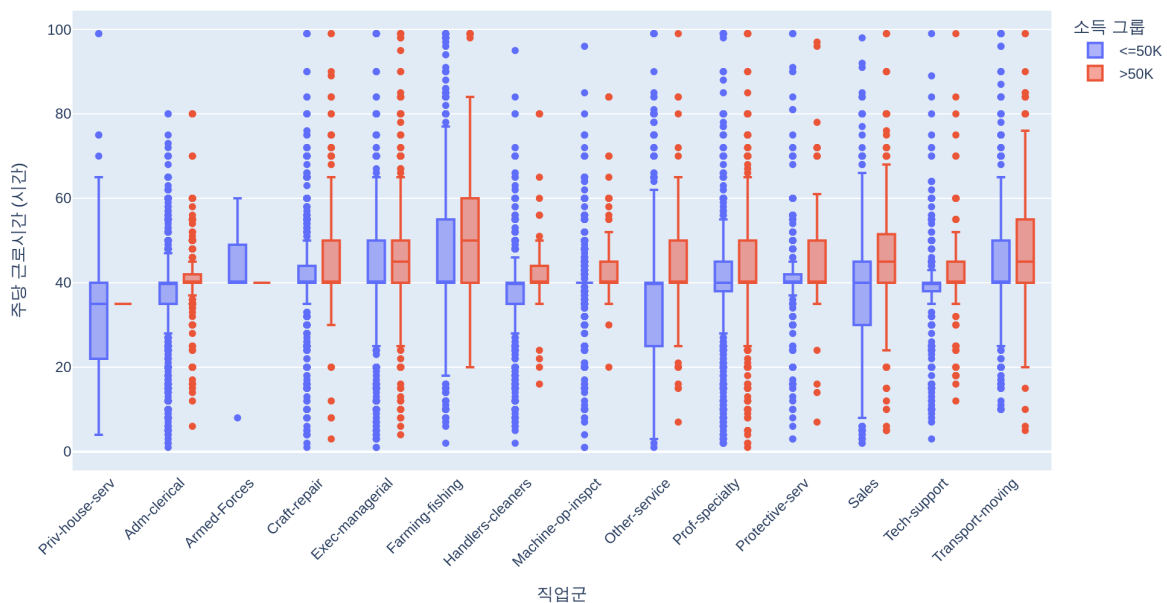
두 소득 그룹 모두 40시간에 강한 스파이크가 있고, **>50K** 그룹이 오른쪽(더 긴 근로시간)으로 이동해 있다. 이 관찰은 아래 6.1절 Welch t-test의 평균차·효과크기와 직접 연결된다.



Plotly 인터랙티브 차트 (그룹비교) - 직업군별 근로시간과 소득

직업군별 분포 차이와 같은 직업 안의 소득 그룹 차이가 함께 보인다. 직업·근로시간·소득의 관계를 인과적으로 분리한 결과는 아니다.

직업군별 주당 근로시간 분포 — 소득 그룹 비교 (n=30,718)



참고 자료(추가 EDA): 소득 그룹별 연령 분포(정적), 수치형 변수 Pearson 상관행렬 히트맵(정적), 교육수준별 고소득 비율(정적), 연령대×교육수준 고소득 비율 히트맵(인터랙티브) 등 4개의 추가 차트를 [EDA.ipynb](#) 에서 함께 작성했다. 교육수준이 높을수록 고소득 비율이 전반적으로 증가하지만 모든 인접 구간에서 엄밀하게 단조증가하지는 않는다.

5. 통계 분석 (H1, H2 검증)

5.1 H1 - 소득 그룹 간 주당 근로시간 (Welch t-test)

항목	>50K	≤50K
n	7,841	24,720
평균 (시간)	45.473	38.8402
표준편차	11.013	12.319

- 평균차 = **6.6328 시간/주** (95% CI [6.3447, 6.9209])
- Welch t(14570.2) = 45.1231, 양측 p < 0.001
- Hedges' g = **0.5519** (중간 효과)

해석: $p < 0.001 < \alpha = 0.05$ 이므로 H_0 를 기각. 고소득 그룹의 주당 근로시간 평균이 유의하게 다르며, 표본이 커서 p-value 자체보다 효과 크기가 중요하다: 그룹 차이는 주당 약 6.6시간으로 실질적으로도 의미 있는 크기다. 이는 **관측 연관성**이며, 근로시간을 늘리면 소득이 오른다는 인과 주장을 지지하지 않는다(교육·직종·연령 등 교란 요인 미통제).

극단값 민감도 (윤저화 재검정): 근로시간을 전체 표본 1%/99% 분위수(8, 80시간)로 윤저화한 뒤 동일한 Welch 검정을 재수행하면 평균차 6.5849 (95% CI [6.3078, 6.862]), Hedges' g = 0.5675, $p < 0.001$ 이다. 기각 결론과 효과 크기가 원 분석과 실질적으로 같으므로, 그룹 차이는 극단값 처리 방식에 안정적이라고 판단한다.

5.2 H2 -상관·연관 (2차 가설군, Holm 보정)

수치형 상관 (Pearson, Holm 보정 p):

변수쌍	n	Pearson r	p(Holm)
age × hours-per-week	32561	0.0688	< 0.001
education-num × hours-per-week	32561	0.1481	< 0.001
education-num × capital-gain	32561	0.1226	< 0.001
age × education-num	32561	0.0365	< 0.001

범주형 연관 (카이제곱 + Cramér's V, 같은 가설군 보정):

변수쌍	n	χ^2	자유도	Cramér's V	p(Holm)
income × education	32561	4429.65	15	0.3688	< 0.001
income × sex	32561	1518.89	1	0.216	< 0.001

해석: 2차 가설군의 Holm 보정 p-value가 모두 α 미만이라 "연관 있음"으로 결론 내린다. 다만 상관계수 크기가 작은 항목($|r| < 0.2$, 예: age×hours-per-week, age×education-num)은 통계적 유의성과 무관하게 실질적 연관은 약하다고 서술한다 -대표본($n=32,561$)에서는 작은 효과도 쉽게 유의해지는 점을 경계한다. 반면 income×education($V=0.3688$)은 범주형 연관 기준으로 중간 이상의 실질적 연관으로 해석할 수 있다.

6. 결측치 대체법 비교 (H4 검증)

6.1 실험 설계

어떤 채우기 방법이 좋은지는 정답을 알아야 채점할 수 있다. 그래서 **값이 전부 채워져 있는 행만 골라, 알고 있는 값을 일부러 지웠다가 각 방법으로 다시 채워 보고 원래 값과 비교**하는 실험을 설계했다. 소득 비율을 유지한 **8,000행** 표본에서(학습용 6,000행 / 평가용 홀드아웃 2,000행), 컬럼 `age`, `hours-per-week`, `workclass`, `occupation` 의 값을 약 15% 지웠다. 지우는 위치는 시드를 바꿔 **10번 다시 뽑아** 평균·표준편차로 보고한다 -한 번의 운 좋은(나쁜) 배치로 우열을 단정하지 않기 위해서다.

- **MCAR 시나리오:** 모든 칸을 같은 확률로 무작위로 지움
- **MAR 시나리오:** $P(\text{결측}) = \text{sigmoid}(b_0 + 0.9 \cdot z(\text{education-num}) - 0.7 \cdot I(\text{capital-gain} > 0))$ -학력이 높고 자본소득이 없는 사람일수록 지워질 확률이 커지도록 만들어, H3에서 진단한 "특성에 따라 비는 결측"을 재현

누수 통제: 모든 대체기(중앙값/최빈값·kNN-MICE)의 통계량과 모형 적합은 train 행에서만 수행하며 홀드아웃 행 값은 대체 계산에 사용되지 않는다. 딥러닝 벤치마크와 동일 조건.

진값을 알고 있으므로 복원 정확도를 직접 측정하며, 홀드아웃 행은 마스킹·대체 대상이 아니고 대체기 적합에도 사용되지 않는다. 동일 표본과 처음 3개 마스크 세트가 `deep_imputation/data/` 로 내보내져 딥러닝 대체가 **같은 조건**에서 평가된다.

6.2 결과 (평균 ± 표준편차, 마스크 반복 기준)

방법	시나리오	표준화 RMSE↓	범주 오분류율↓	다운스트림 F1↑	평균 소요(s)	반복
완전제거법 (complete-case)	MAR			0.6337 ± 0.0152	0.02	10
중앙값/최빈값 (SimpleImputer)	MAR	0.9927 ± 0.0229	0.6126 ± 0.0144	0.6527 ± 0.0048	0.03	10
kNN (KNNImputer, k=5)	MAR	0.9335 ± 0.0123	0.706 ± 0.0155	0.646 ± 0.0032	0.32	10

방법	시나리오	표준화 RMSE↓	범주 오분류율↓	다운스트림 F1↑	평균 소요(s)	반복
MICE (IterativeImputer)	MAR	0.9243 ± 0.0179	0.6435 ± 0.0103	0.6492 ± 0.005	0.08	10
완전제거법 (complete-case)	MCAR			0.6443 ± 0.0071	0.02	10
중앙값/최빈값 (SimpleImputer)	MCAR	1.0008 ± 0.0157	0.5583 ± 0.0129	0.6376 ± 0.0057	0.03	10
kNN (KNNImputer, k=5)	MCAR	0.9247 ± 0.0222	0.678 ± 0.0091	0.6452 ± 0.0043	0.36	10
MICE (IterativeImputer)	MCAR	0.9364 ± 0.0165	0.6011 ± 0.0103	0.6495 ± 0.0028	0.08	10
CACTI-style (PyTorch)	MAR	0.8498 ± 0.0108	0.4587 ± 0.0113	0.6335 ± 0.0033	110.3003	3
CACTI-style (PyTorch)	MCAR	0.8552 ± 0.0137	0.4642 ± 0.0078	0.6386 ± 0.0044	91.509	3
TabNAT-style (PyTorch)	MAR	1.2366 ± 0.1183	0.47 ± 0.0068	0.638 ± 0.0063	83.483	3
TabNAT-style (PyTorch)	MCAR	1.2978 ± 0.1309	0.4644 ± 0.0104	0.6401 ± 0.0045	95.6027	3

딤러닝 결과는 `deep_imputation/` 패키지 실행 산출물(`dl_results.json`, device=cpu, torch 2.13.0, 마스크 반복 3회)을 병합한 것이다. 패키지는 GPU 서버·로컬(CPU/Apple Silicon) 어디서든 동일하게 실행된다.

H4 판정 (rep 단위 대응 비교):

- **H4a · 수치형 RMSE (MAR):** MICE-단순대체 평균 차이 -0.0684 ± 0.0076 , MICE 우위 10/10회
- **H4a · 수치형 RMSE (MCAR):** MICE-단순대체 평균 차이 -0.0644 ± 0.0084 , MICE 우위 10/10회
- **범주형 오분류율 (MAR):** MICE-단순대체 평균 차이 $+0.0309 \pm 0.014$, MICE 우위 0/10회 -서수 인코딩+반올림 근사의 한계로 최빈값 대체가 오히려 낮다
- **범주형 오분류율 (MCAR):** MICE-단순대체 평균 차이 $+0.0429 \pm 0.0056$, MICE 우위 0/10회 -서수 인코딩+반올림 근사의 한계로 최빈값 대체가 오히려 낮다
- **다운스트림 F1 (MAR):** MICE-단순대체 평균 차이 -0.0035 ± 0.0025 , MICE 우위 1/10회
- **다운스트림 F1 (MCAR):** MICE-단순대체 평균 차이 $+0.0119 \pm 0.0055$, MICE 우위 10/10회
- **H4b · 시나리오 간 격차 변화:** (단순대체-MICE) RMSE 격차의 MAR-MCAR 차이 = $+0.0040$ (95% CI $[-0.0047, 0.0127]$, 대응 $t(9) = 1.046$, $p = 0.3228$, 양수 6/10회) → **CI가 0을 포함해 H4b는 지지되지 않음** (격차 축소를 주장할 수 없다)

요약하면, “MICE 우위”는 수치형 RMSE에 한정된 결론이다. 범주형 복원은 본 구현의 서수 인코딩 근사 때문에 최빈값 대체보다 나쁘고, 다운스트림 F1 차이는 작으며 시나리오 간 격차 축소(H4b)는 반복 변동성 안에서 구별되지 않았다.

딤러닝 비교 (요약): CACTI-style은 RMSE에서 MICE를 앞섰으나 TabNAT-style은 그렇지 않았다 -상세 수치는 아래 요약 박스 참고.

해석의 한계를 먼저 명시한다. 위 비교는 단일 아키텍처 구현과 소수(3회) 마스크 반복에 기반한 것으로, 통계적 우열 단정이 아니라 경향 보고다. 또한 본 구현의 컬럼 컨텍스트는 열 이름의 해시 기반 고정 벡터 + 학습 임베딩으로, 원 논문 CACTI의 **언어모델 의미 임베딩이 아니다**. 따라서 성능 차이를 “컨텍스트가 변수 의미를 활용한 결과”로 귀속할 수 없으며, 원인 규명에는 구성요소별 ablation(copy masking 제거, 컨텍스트 제거 등)이 필요하다. TabNAT-style의 표준화 RMSE 열세의 가능한 설명 중 하나는 지표 특성이다: 조건부 분포에서 표본을 추출하는 생성형 대체는 조건부 평균을 출력하는 점추정 대체보다 제곱오차 지표에서 불리할 수 있다(표본 분산이 오차에 가산). 다만 학습 부족·아키텍처 축소 등 다른 원인을 배제하지 못했으므로 단정하지 않는다. 다운스트림 F1은 딤러닝이 MICE 대비 **작지만 전반적으로 낮았다** (동일 마스크 대응 비교 12건 중 11건에서 낮음, 평균 차이 -0.0105). 차이의 크기는 1%p 안팎으로 작으나 방향은 일관적이며, 복원 정확도의 이득이 예측 성능으로 자동 번역되지 않는다는 사전 예상(H4c)과 부합한다.

🔗 딤러닝 기반 대체법(CACTI-style, TabNAT-style) 상세 비교는 이번 실험 범위를 넘어서는 심화 실험이라 요약만 남긴다. 같은 마스크로 비교했을 때 복원 오차(RMSE)는 CACTI-style이 MICE보다 낮았지만, 다운스트림 F1은 오히려 소폭 낮게 나왔다(동일 마스크 대응 비교 평균 차이 -0.0105) -“복원 정확도의 개선이 예측 성능 개선으로 자동 번역되지 않는다”는 예상(H4c)과 부합한다. 구현 세부사항(Copy Masking, 컬럼 컨텍스트 인코딩 등)과 관련 논문(CACTI: Gorla et al. ICML 2025, TabDAR: arXiv:2410.21523)은 `deep_imputation/` 패키지 문서에 남겨두고, 본 보고서는 실험 범위인 고전적 대체법(완전제거·SimpleImputer·kNN·MICE) 비교에 집중한다.

결론만 요약: 이 데이터 규모(수만 행)에서는 통계적 추론 목적이라면 불확실성을 Rubin's rules로 전파할 수 있는 MICE 계열이 여전히 정석이다.

7. 완전제거 공유본 기준 -5개 분류 모델 통일 비교

이 장은 모델만 다른 공정한 비교를 위해 결측행을 제거한 공통 train/test를 사용한다. 결측을 “Unknown”으로 보존하는 9장의 최종 통합 코드와는 목적·표본·시드가 다르다.

7.1 실험 설계 -비교 조건을 왜 다시 통일했는가

각 팀원이 개별적으로 실행한 최초 스크립트(`notebooks/legacy/`)는 시드(LR/RF/XGB는 42, NN은 20260807, SVC는 미지정), 피쳐셋, 전처리, 평가 지표가 모두 달라 표 하나로 나란히 놓고 비교할 수 없었다. 이에 다음 4가지를 통일한 재실험을 별도로 수행했다.

- **데이터:** `adult_train_dropna.csv` (30,162행) / `adult_test_dropna.csv` (15,060행) -결측행 완전제거본, train 양성비율 24.89% · test 24.57%
- **시드:** 전 모델 공통 `20260807`

- 피쳐셋: 수치형 5개(age , education-num , capital-gain , capital-loss , hours-per-week) + 범주형 8개(workclass , education , marital-status , occupation , relationship , race , sex , native-country) , fnlwgt 제외(표본가중치라 예측 변수 부적합) -원핫 인코딩 후 103차원
- 전처리: ColumnTransformer (수치형 StandardScaler + 범주형 OneHotEncoder(handle_unknown="ignore"))를 모델마다 새로 적합해 fit 상태가 공유되지 않도록 함
- 주 평가지표: F1 -양성 클래스(>50K)가 약 25%인 불균형 데이터라 accuracy는 변별력이 떨어지므로, ROC-AUC를 보조 지표로 병기

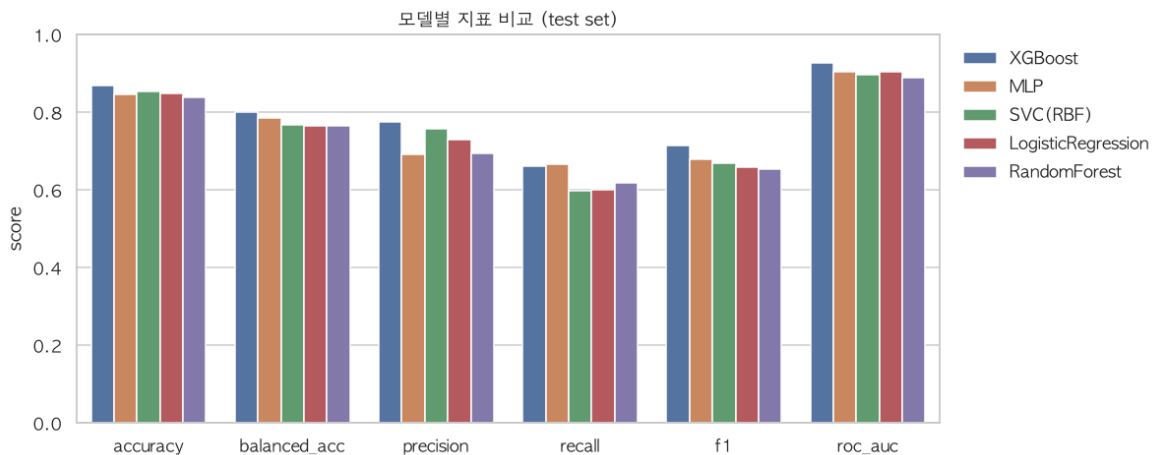
7.2 완전제거 공유본 비교 결과 (adult_test_dropna, n=15,060)

모델	Accuracy	Balanced Acc	Precision	Recall	F1	ROC-AUC	학습시간(s)
XGBoost	0.8695	0.7992	0.7750	0.6608	0.7133	0.9270	0.51
MLP (Neural Network)	0.8446	0.7845	0.6905	0.6662	0.6781	0.9040	1.48
SVC(RBF)	0.8538	0.7675	0.7560	0.5978	0.6677	0.8977	10.26
LogisticRegression	0.8471	0.7638	0.7295	0.6000	0.6585	0.9030	0.68
RandomForest	0.8392	0.7648	0.6936	0.6186	0.6540	0.8881	2.53

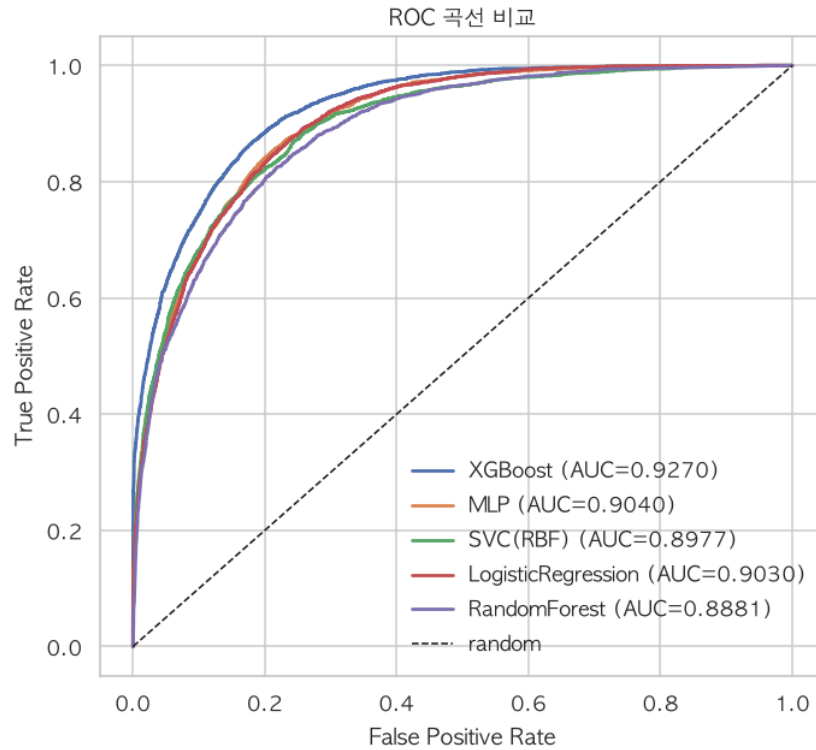
F1 기준으로 XGBoost가 5개 모델 중 가장 높고(0.7133), ROC-AUC(0.9270)와 학습 시간(0.51초) 모두에서도 최상위다 -성과와 비용 두 축 모두에서 지배적이다. 흥미로운 점은 트리 앙상블인 RandomForest가 오히려 5개 중 F1이 가장 낮다는 것으로, 트리 기반이라는 사실 자체가 성능을 보장하지 않으며 부스팅(XGBoost)과 배깅(RandomForest)의 차이가 이 데이터에서는 상당함을 보여준다. SVC(RBF)는 precision(0.7560)은 두 번째로 높지만 recall(0.5978)이 가장 낮고, 학습 시간도 10.26초로 다른 모델 대비 7~20배 길다 -커널 기반 방법이 정형 데이터-저차원 피쳐(103차원)에서 트리 계열 대비 시간 대비 이득이 낮다는 근거로 볼 수 있다.

7.3 시각화

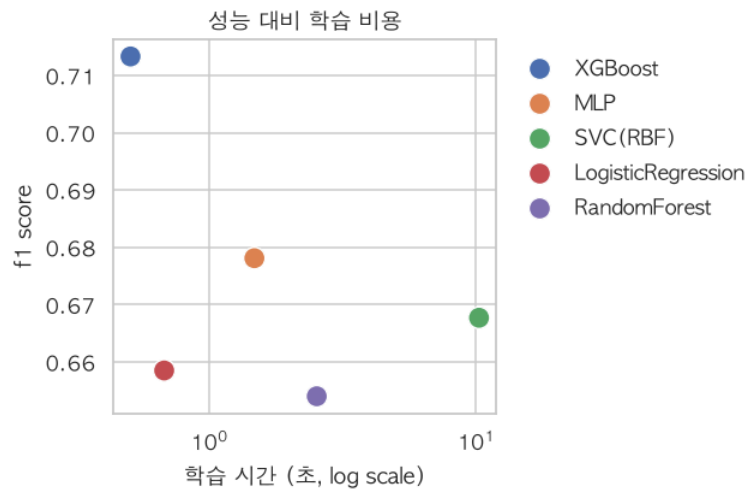
- 6개 지표(accuracy, balanced_acc, precision, recall, f1, roc_auc)를 모델별로 묶은 막대그래프



- 5개 모델의 ROC 곡선을 한 그림에 겹쳐 그린 비교 (모두 AUC 0.88 이상으로 순위 구분선이 촘촘함)



- 학습 시간(로그축) 대 F1의 산점도 -XGBoost가 좌상단(빠르고 정확한) 영역에 단독 위치



7.4 Neural Network (MLP)

- 구성: `MLPClassifier(hidden_layer_sizes=(64,32), activation="relu", solver="adam", alpha=1e-4, early_stopping=True, random_state=20260807)`
- 결과: F1 0.6781, ROC-AUC 0.9040, 학습시간 1.48초 -5개 모델 중 2위
- 인사이트: recall(0.6662)이 XGBoost(0.6608)보다도 높아 5개 모델 중 최고치를 기록했다. 즉 고소득자를 '놓치지 않는' 능력은 가장 좋지만, precision(0.6905)이 상대적으로 낮아 그만큼 거짓 양성도 늘어난다. 얇은 2계층 구조(64 → 32)임에도 트리 계열과 견줄 만한 성능이 나온 것은, 원핫 인코딩된 103차원 입력이 신경망이 다루기에 과도하게 복잡하지 않은 규모이기 때문으로 보인다.
- 개선 방향: 은닉층 크기·`alpha` (L2 정규화 강도)를 `GridSearchCV`로 탐색하지 않고 legacy 설정값을 그대로 썼다. `class_weight`에 대응하는 `sample_weight`를 적용하면 precision-recall 균형을 조정할 여지가 있다.

모델	Accuracy	Bal.Acc	Precision	Recall	F1	ROC-AUC
DummyClassifier (최빈 클래스, 참고용)	0.7543	0.5000	0.0000	0.0000	0.0000	0.5000
Neural Network (MLPClassifier)	0.8514	0.7824	0.7199	0.6468	0.6814	0.9089

7.5 SVC (RBF Kernel)

- **구성:** `SVC(kernel="rbf", C=1.0, gamma="scale", random_state=20260807).probability=True` (내부 5-fold Platt 스케일링)를 켜면 학습이 몇 배 느려지므로, 확률 대신 `decision_function` 점수로 ROC-AUC를 계산했다 -AUC는 순위 기반 지표라 단조 변환에 불변이므로 확률로 계산한 값과 같다.
- **결과:** F1 0.6677, ROC-AUC 0.8977, 학습시간 10.26초 -5개 모델 중 학습이 가장 오래 걸렸다(2위 대비 약 4배, XGBoost 대비 약 20배).
- **인사이트:** precision(0.7560)은 XGBoost 다음으로 높아 "양성으로 예측하면 맞을 확률"은 준수하지만, recall(0.5978)이 5개 모델 중 가장 낮아 실제 고소득자를 놓치는 비율이 크다. 커널 서포트벡터머신은 이론적으로 비선형 결정경계를 유연하게 잡을 수 있지만, 이번처럼 원핫 인코딩으로 차원이 늘어난(103차원) 희소 데이터에서는 그 유연성이 학습 비용 대비 성능 이득으로 잘 이어지지 않았다.
- **개선 방향:** `class_weight="balanced"` 로 불균형을 보정하면 recall이 오를 가능성이 있고, `kernel="linear"` 나 `LinearSVC` 로 바꾸면 이 정도 규모에서는 학습 시간을 크게 줄이면서 성능 손실은 크지 않을 수 있다(선형 결정경계로도 로지스틱 회귀가 이미 F1 0.6585를 내고 있어, 비선형 커널의 실익이 이 데이터에서 크지 않을 가능성을 시사한다).

SVC 최종 결과	
Training Time	: 10.95 sec
Accuracy	: 0.8545
Precision	: 0.7578
Recall	: 0.5995
F1 Score	: 0.6694

7.6 Random Forest & Logistic Regression

Logistic Regression - `LogisticRegression(max_iter=5000, solver="saga", random_state=20260807)`

- 결과: F1 0.6585, ROC-AUC 0.9030, 학습시간 0.68초
- 5개 모델 중 가장 단순한 선형 모델임에도 ROC-AUC는 0.9030으로 XGBoost(0.9270), MLP(0.9040)에 이어 3위이며 RandomForest(0.8881)보다도 높다. 즉 이 데이터는 선형 결정경계만으로도 순위 판별력(AUC)은 상당히 확보되고, F1 격차는 주로 임계값 0.5에서의 분류 성능 차이에서 온다.
- **참고:** 이 결과는 전처리 노트북(`preprocessing-2.ipynb`)에서 별도로 검증한 **결측치 대체(imputation)** 기반 최종 Pipeline의 로지스틱 회귀 성능(test F1 0.6509, ROC-AUC 0.9036, `adult.test` 원본 n=16,281)과 거의 일치한다. 즉 이 특정 모델·피처 조합에서는 **결측행을 완전제거했는지, 중앙값/최빈값으로 대체했는지**가 **최종 예측 성능에 미치는 영향이 미미했다** -4장 H4에서 이미 관찰한 "복원 정확도의 이득이 예측 성능으로 자동 번역되지 않는다"는 결론과 같은 방향이다.

Random Forest - `RandomForestClassifier(n_estimators=250, random_state=20260807, n_jobs=-1)`

- 결과: F1 0.6540, ROC-AUC 0.8881, 학습시간 2.53초
- 5개 모델 중 F1이 가장 낮았다. 같은 트리 기반이라도 부스팅(XGBoost, F1 0.7133)과 배깅(RandomForest, F1 0.6540)의 격차가 0.06 가까이 벌어진 것은, 이 데이터·피처셋 조합에서는 오차를 순차적으로 보정해나가는 부스팅 방식이 병렬 독립 트리를 평균내는 배깅 방식보다 유리했음을 시사한다.
- **개선 방향:** `n_estimators` · `max_depth` · `min_samples_leaf` 등 하이퍼파라미터는 legacy 스크립트의 수동 설정값을 그대로 가져온 것이라, `GridSearchCV` 로 탐색하면 XGBoost와의 격차가 줄어들 여지가 있다.

```

~/team-analysis P feat/temp-ml-test ? @ 16:38
> PYTHONPATH=src uv run python notebooks/ml_random_forest_dropna_test.py

RandomForest 성능 요약 (train → test)
Dataset Accuracy Precision Recall F1 ROC-AUC
adult_test_dropna 0.8461 0.7158 0.6195 0.6642 0.8992

~/team-analysis P feat/temp-ml-test ? @ 16:38
> PYTHONPATH=src uv run python notebooks/ml_logistic_regression_dropna_test.py

LogisticRegression 성능 요약 (train → test)
Dataset Accuracy Precision Recall F1 ROC-AUC
adult_test_dropna 0.8478 0.7299 0.6041 0.661 0.9033

```

7.7 XGBoost -완전제거 비교 실험 선정 모델

- 구성: 하이퍼파라미터는 legacy 실험 스크립트 값을 시드만 20260807 로 통일해 그대로 사용
- 비교 실험 내 선정 근거: F1 0.7133으로 5개 모델 중 최고, ROC-AUC(0.9270)·학습시간(0.51초)도 모두 최상위였다. 이 평가는 완전제거 공유본 조건에 한정된다.

테스트셋 분류 리포트 (adult_test_dropna.csv, n=15,060)

클래스	Precision	Recall	F1	Support
≤50K	0.89	0.94	0.92	11,360
>50K	0.77	0.66	0.71	3,700
accuracy			0.87	15,060
macro avg	0.83	0.80	0.81	15,060
weighted avg	0.87	0.87	0.87	15,060

다수 클래스(≤50K)는 recall 0.94로 매우 잘 잡아내지만, 소수 클래스(>50K)는 recall 0.66에 그쳐 고소득자 3명 중 1명 정도는 놓친다. 이는 5개 모델 공통의 경향(불균형 데이터에서 소수 클래스 recall이 다수 클래스보다 낮음)이며 XGBoost도 예외는 아니다.

- 모델 저장·재로드 검증: `joblib.dump()` 로 저장 후 다시 불러와 예측을 비교 -완전히 일치함을 확인(assert 통과)

```

[train] {'dataset': 'adult_train_dropna', 'accuracy': 0.888402625820569,
'precision': 0.8208862720793306, 'recall': 0.7056473095364945, 'f1':
0.7589170605930382, 'roc_auc': 0.9476947052363416}
[test] {'dataset': 'adult_test_dropna', 'accuracy': 0.8692563081009296,
'precision': 0.774500475737393, 'recall': 0.66, 'f1': 0.7126805778491172, 'roc_auc':
0.9260007375333079}

```

7.8 통일 실험이 보여준 것

- 시드·피처셋·전처리·지표를 통일하기 전에는 팀원 개인 스크립트마다 결과가 달라 "어떤 모델이 더 나은가"를 답할 수 없었다. **비교 가능한 조건을 먼저 맞추는 것이** 개별 모델 성능을 올리는 것보다 선행 조건이었다는 것이 이번 재실험의 가장 큰 교훈이다.
- F1을 주 지표로 선택한 것 자체가 결론에 영향을 준다. accuracy를 주 지표로 삼았다면 XGBoost(0.8695)와 SVC(0.8538)의 격차가 F1 기준 보다 좁게 보였을 것이고, 다수 클래스 비율(75.4%)에 가까운 값들이라 모델 간 변별력이 낮아 보이는 착시가 생겼을 것이다.
- 로지스틱 회귀의 ROC-AUC가 완전제거본과 대체본에서 비슷하게 관측됐지만, 두 결과는 표본과 평가 절차가 달라 전처리 방식의 영향이 작다고 단정할 수 없다. 완전제거 후 평균 근로시간·고소득 비율·남성 비율이 같은 방향으로 변한 점을 고려하면 선택편향 가능성도 함께 남는다.

개인적으로 해보고 싶은 후속 과제

- 하이퍼파라미터는 legacy 스크립트의 수동 설정값이므로, 선정 모델(XGBoost)에 대해 `GridSearchCV` / `RandomizedSearchCV` 로 탐색하면 개선 여지가 있다.
- 현재는 단일 train/test 분할 기준이므로, 교차검증으로 지표의 분산까지 함께 보면 더 견고한 비교가 된다.
- 불균형(양성 약 25%)에 대해 `class_weight` 나 분류 임계값 조정을 적용하면 소수 클래스(>50K) recall을 끌어올릴 수 있다.
- 7장은 완전제거 공유본 기준, 3장·6장은 원본 결측을 보존한 진단·대체 실험 기준, 9장은 `Unknown` 보존과 시드 42를 적용한 최종 코드 기준이다. 세 결과는 목적이 다르며 직접적인 우열 비교로 사용하지 않는다.

8. 팀 회고 및 다음 단계

- 이번 실습에서 팀이 잘한 점: 개별적으로 흩어져 있던 실험(legacy 스크립트)을 시드·피처·전처리·지표 기준으로 통일해, 서로 다른 사람이 만든 모델을 공정하게 비교할 수 있는 표 하나로 수렴시켰다.

- 가장 크게 배운 점: "어떤 모델이 좋은가"라는 질문은 "어떤 데이터와 전처리·시드·평가 지표에서 비교했는가"와 분리할 수 없다. 완전제거 공유본과 최종 통합 코드의 수치는 각자의 조건 안에서만 해석해야 한다.
- 다음 프로젝트에서 시도해보고 싶은 것: 선정 모델(XGBoost)에 대한 하이퍼파라미터 탐색(GridSearchCV), K-Fold 교차검증으로 지표 분산까지 확인, 불균형 보정(class_weight/임계값 조정)으로 소수 클래스 recall 개선, 그리고 dropna 기준 실험과 imputation 기준 실험(3장·6장)을 완전히 동일한 조건으로 재통일하는 것.

9. 최종 통합 코드 실행 결과 (scala-python-day2)

이 장은 `src/main.py` 전체 파이프라인의 실제 실행 결과다. 최종 코드는 중복 24행만 제거하고 범주형 결측을 "Unknown"으로 보존했으며, 모든 확률적 단계에 `RANDOM_STATE=42`를 적용했다. 이는 `workclass`·`occupation` 결측이 함께 발생하는 구조와 완전제거 후 표본 구성 변화가 확인된 점을 반영한 최종 선택 결과이다.

7장의 완전제거·시드 20260807 결과는 팀 공통 모델 비교 실험, 이 장은 제출 코드의 공식 실행 결과다. 두 수치는 평가 조건이 다르므로 직접 비교하지 않는다.

9.1 실행 환경

항목	값
실행 명령	<code>python src/main.py</code>
종료 코드	0 (정상 완료)
원본 데이터	<code>adult.data</code> 32,561행 × 15열
pytest	23 passed (0 failed)
ruff check	All checks passed!
Git 커밋 이력	21개 커밋 (feature 브랜치 병합 포함)

9.2 데이터 준비 -Pandas vs Polars

라이브러리	행 수	컬럼 수	로딩 시간(초, 3회 중 최소)
Pandas	32,561	15	0.0871
Polars	32,561	15	0.0316

결과 형태 일치: True / 더 빠른 쪽: Polars (2.76배)

9.3 결측치·중복 처리

항목	값
처리 전 → 후 행 수	32,561 → 32,537
제거된 중복 행	24
결측 포함 행	2,399행 (7.37%)
처리 방식	삭제하지 않고 "Unknown" 범주로 대체 (결측 셀 4,262개)

컬럼별 결측: `workclass` 1,836 / `occupation` 1,843 / `native-country` 583

9.4 기술통계·상관분석 (핵심만)

변수	평균	표준편차	왜도
age	38.586	13.638	0.558
education-num	10.082	2.572	-0.310
capital-gain	1078.444	7387.957	11.949
hours-per-week	40.440	12.347	0.229

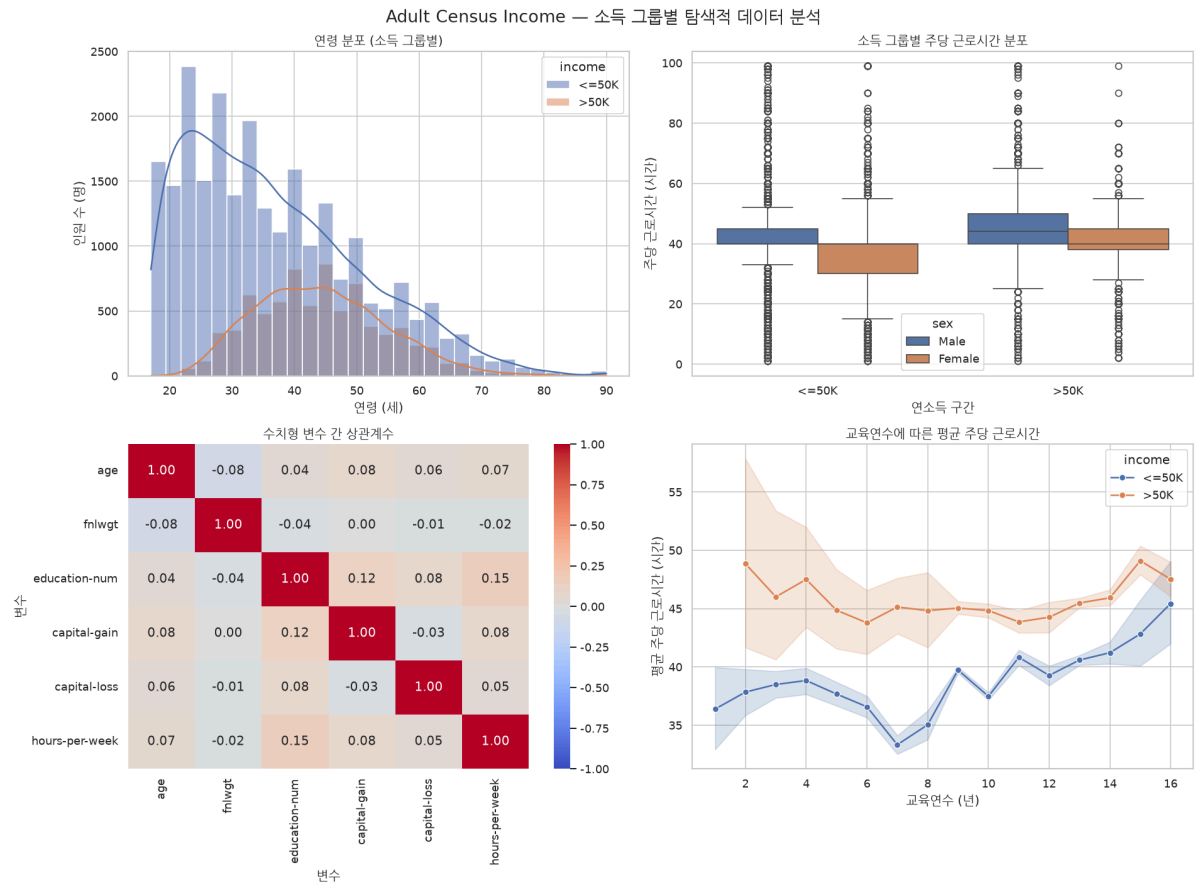
최고 상관쌍: `education-num` ↔ `hours-per-week` ($r=+0.148$, 약한 양의 상관)

소득 그룹별 평균 차이: age +7.46 / education-num +2.01 / capital-gain +3858.28 / hours-per-week +6.63 (모두 >50K 그룹이 높음)

9.5 시각화

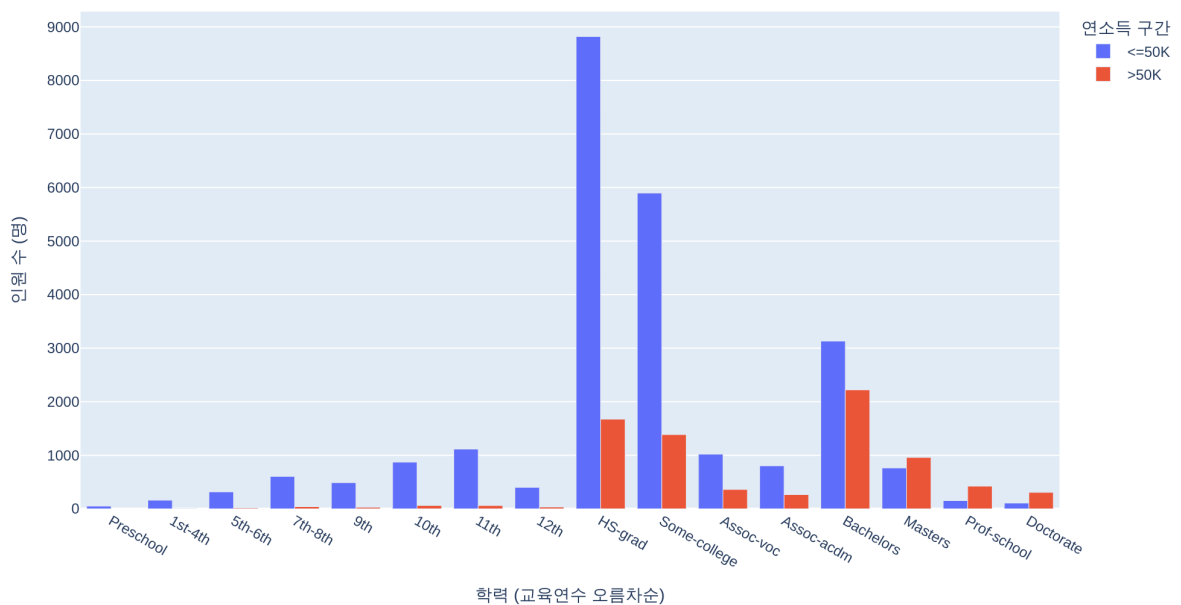
Seaborn 정적 차트 (2×2, 분포+그룹비교+상관관계 통합) -실제 실행 결과:

(연령 분포(그룹별) · 성별×소득 근로시간 박스플롯 · 상관행렬 히트맵 · 교육연수별 근로시간 추이, 4종을 한 figure에 배치. 파일: `outputs/seaborn_eda.png`)



Plotly 인터랙티브 차트 -학력별 소득구간 인원분포 (그룹비교, 막대차트, [outputs/plotly_education_income.html](#))

학력별 소득 구간 인원 분포 (Adult Census Income)



9.6 통계 검정

Welch t-test (소득 그룹 간 평균 차이, $\alpha=0.05$):

변수	≤50K 평균	>50K 평균	t	p-value	유의성
age	36.79	44.25	50.235	<0.001	유의
fnlwgt	190345.93	188000.48	-1.748	0.081	유의하지 않음
education-num	9.60	11.61	64.876	<0.001	유의
capital-gain	148.88	4007.16	23.426	<0.001	유의
capital-loss	53.19	195.05	20.233	<0.001	유의
hours-per-week	38.84	45.47	45.095	<0.001	유의

→ 6개 중 5개 변수가 유의. **fnlwgt** (표본가중치)만 유의하지 않은 것은 이 변수가 애초에 예측 변수로 부적합하다고 판단해 ML 피처에서 제외한 결정과 일관된다.

카이제곱 독립성 검정 (**sex** × **income**): $\chi^2=1516.540$, $df=1$, $p<0.001$ → 성별과 소득구간은 독립이 아니다(연관 있음).

9.7 ML Pipeline -5개 모델 비교

학습 데이터	테스트 데이터	전처리 후 피처 수			
26,029행	6,508행	107개			
모델	Accuracy	Precision	Recall	F1	ROC-AUC
---	---	---	---	---	---
XGBoost	0.8774	0.7831	0.6792	0.7275	0.9316
RandomForest	0.8722	0.7884	0.6416	0.7075	0.9219
MLP	0.8612	0.7343	0.6645	0.6977	0.9181
LogisticRegression	0.8568	0.7387	0.6276	0.6786	0.9114
SVC(RBF)	0.8605	0.7687	0.6020	0.6753	0.9085

선정 모델: XGBoost (다수 클래스 베이스라인 정확도 0.7591 대비 +0.1183)

클래스	Precision	Recall	F1	Support
≤50K	0.90	0.94	0.92	4,940
>50K	0.78	0.68	0.73	1,568
accuracy			0.88	6,508

모델 저장·재로딩 검증: `outputs/income_pipeline.pkl` -통과

9.8 공정성(민감 속성) 점검 -가이드라인 이상의 추가 분석

속성	집단	표본수	실제 고소득률	재현율	정밀도
sex	Female	2,130	0.1192	0.6417	0.8030
sex	Male	4,378	0.3001	0.6865	0.7796
race	Black	644	0.1351	0.5977	0.8387
race	White	5,525	0.2557	0.6865	0.7854
race	Amer-Indian-Eskimo	66	0.1364	0.4444	0.6667

Female-Black 집단의 재현율이 Male-White 대비 낮게 나와, 모델이 학습 데이터에 이미 존재하는 격차를 그대로 재현하고 있음을 보여준다(가이드라인에는 없는 자체 추가 분석).

9.9 report.md 자동 생성

`src/report.py` + Jinja2 템플릿(`src/templates/report.md.j2`)으로 위 9.1~9.8의 모든 수치가 **코드 실행 시점에 자동으로 채워져** `outputs/report.md` (8,913자)로 생성됨을 확인했다. 수기 입력된 숫자 없음.

9.10 종합 결론

아래 결론과 모델 지표는 최종 통합 코드(`Unknown` 보존, `RANDOM_STATE=42`) 실행을 기준으로 한다.

- Pandas 대비 Polars가 로딩 2.76배 빠름을 확인했으나, 실무 판단은 데이터 규모·생태계 호환성 등 다른 기준이 우선한다.
- 결측은 삭제 대신 `Unknown` 범주로 보존하는 편이 이 데이터의 구조적 결측 패턴(H3)에 더 적합하다고 최종 판단했다.
- 6개 수치형 변수 중 5개에서 소득 그룹 간 평균 차이가 통계적으로 유의했다.
- 5개 후보 모델 중 **XGBoost**가 F1(0.7275)·ROC-AUC(0.9316) 모두 최고로 최종 선정되었다.
- **한계:** 하이퍼파라미터 미탐색(GridSearchCV 등), 1994년 데이터라 현재 소득 구조에 그대로 적용 불가, 민감 속성 간 성능 격차는 데이터 자체의 편향을 모델이 재현한 결과.